

RELATORIO RESUMIDO - CAMERAS DA API STEALTH LENS

Data: 19/05/2026

OBJETIVO

Foi implementado na pasta 'API' um modulo de cameras seguindo o padrao da API 'ping-health': separacao em 'domain', 'application', 'repository', 'http', 'infra' e 'httpx'. O foco foi deixar a base pronta para cadastrar, listar, atualizar e remover cameras de forma autenticada e organizada.

O QUE FOI IMPLEMENTADO

Rotas protegidas por JWT:

- 'POST /cameras': cria camera.
- 'GET /cameras': lista cameras do usuario autenticado.
- 'GET /cameras/{id}': busca camera por ID.
- 'PUT /cameras/{id}': atualiza camera.
- 'DELETE /cameras/{id}': remove camera.

O middleware JWT valida o cookie 'access_token', extrai o 'user_id' do token e coloca esse valor no contexto da requisicao. Todas as operacoes de camera usam esse 'user_id', entao um usuario nao consegue acessar cameras de outro usuario.

MODEL PERSISTIDO NO BANCO

Arquivo: 'internal/domain/camera/model.go'

```
type Camera struct {
    ID      uuid.UUID
    Name    string
    Location string
    URL     string
    Status  string
    UserID  uuid.UUID
    User    user.User
    CreatedAt time.Time
    UpdatedAt time.Time
}
```

Tabela criada pelo GORM: 'cameras'

Campos principais:

- 'id': UUID da camera.
- 'name': nome da camera.
- 'location': localizacao.
- 'url': IP ou URL da camera.
- 'status': 'unknown', 'online' ou 'offline'.
- 'user_id': dono da camera.
- 'created_at' e 'updated_at': datas automaticas.

Relacionamento: uma 'Camera' pertence a um 'User', e um 'User' pode ter varias cameras.

CAMADAS CRIADAS

Application

Arquivos:

- 'internal/application/camera/dto.go'
- 'internal/application/camera/service.go'

Metodos principais:

- 'CreateCameraService'
- 'ListCamerasService'
- 'GetCameraByIDService'
- 'UpdateCameraService'
- 'DeleteCameraService'

Validacoes:

- nome e localizacao obrigatorios;
- fonte da camera obrigatoria;
- aceita IP puro;
- aceita 'rtsp', 'rtsps', 'http' e 'https';
- status limitado a 'unknown', 'online' e 'offline'.

Repository

Arquivo: 'internal/repository/camera_repository.go'

Metodos:

- 'CreateCamera'
- 'GetCamerasByUser'
- 'GetCameraByID'
- 'UpdateCamera'
- 'DeleteCamera'

As consultas sensiveis filtram por 'camera_id' e 'user_id', garantindo isolamento por usuario.

HTTP Handler

Arquivo: 'internal/http/handlers/camera.go'

Metodos:

- 'CreateCameraHandler'
- 'ListCamerasHandler'
- 'GetCameraHandler'
- 'UpdateCameraHandler'
- 'DeleteCameraHandler'

Responsabilidade: receber JSON, recuperar usuario autenticado, validar ID da camera, chamar o service e retornar a resposta HTTP.

BANCO E MIGRATIONS

Arquivo: 'internal/infra/database/migration.go'

Ao iniciar a API, o sistema executa:

```
db.AutoMigrate(&user.User{}, &camera.Camera{})
```

Com isso, o GORM cria ou atualiza as tabelas 'users' e 'cameras' no PostgreSQL.

RESULTADO

O modulo de cameras ficou funcional, autenticado e alinhado ao estilo da 'ping-health'. A base agora esta pronta para evoluir para status real online/offline, descoberta automatica de cameras, integracao com o agente local e dashboards.